

Object Oriented Programming

OOPs = Object oriented programming structure

OOPs is a programming model or paradigm which revolves around the concept of "OBJECTS".

An **object** refers to the instance of the class, which contains the instance of the members and behaviors defined in the class template. In the real world, an object is an actual entity to which a user interacts, whereas class is just the blueprint for that object. So the objects consume space and have some characteristic behavior.

A **class** can be understood as a template or a blueprint, which contains some values, known as member data or member, and some set of rules, known as behaviors or functions. So when an object is created, it automatically takes the data and functions that are defined in the class.

It deals with internal designing not external. So it is just to help the programmer to prevent data from mishandling.

Simula is considered the first object-oriented programming language. The programming paradigm where everything is represented as an object is known as a truly object-oriented programming language.

There are several benefits to using object-oriented programming (OOP) in software development:

Modularity: OOP allows you to break a large and complex problem down into smaller and more manageable pieces, or "objects." This makes it easier to write, test, and maintain your code.

Reusability: OOP allows you to reuse code by creating new objects that are similar to existing ones. This can save time and effort when developing software.

Extensibility: OOP allows you to easily add new features to an existing codebase by creating new objects or modifying existing ones.

Maintainability: OOP makes it easier to track and fix errors in your code, as objects are self-contained and can be isolated and tested more easily than procedural code.

Scalability: OOP allows you to build software that can grow and evolve over time, as new objects can be created or added to the existing codebase as needed.

Oops, are based on a bottom-up approach, unlike the structural programming paradigm, which uses a top-down approach.

What are some other programming paradigms other than OOPs?

Programming paradigms refers to the method of classification of programming languages based on their features. There are mainly two types of Programming Paradigms:

Imperative Programming Paradigm

Declarative Programming Paradigm

Now, these paradigms can be further classified based:

1. Imperative Programming Paradigm: Imperative programming focuses on HOW to execute program logic and defines control flow as statements that change a program state. This can be further classified as:

a) Procedural Programming Paradigm: Procedural programming specifies the steps a program must take to reach the desired state, usually read in order from top to bottom.

b) Object-Oriented Programming or OOP: Object-oriented programming (OOP) organizes programs as objects that contain some data and have some behavior.

c) Parallel Programming: Parallel programming paradigm breaks a task into subtasks and focuses on executing them simultaneously at the same time.

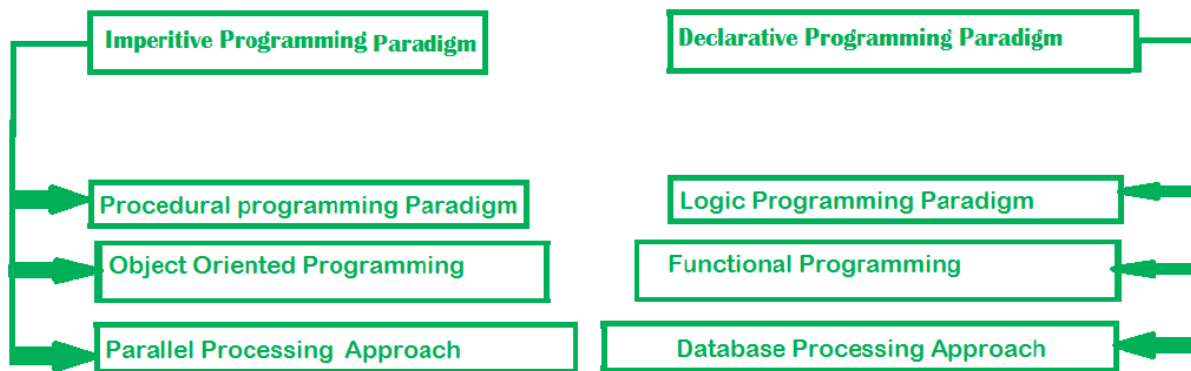
2. Declarative Programming Paradigm: Declarative programming focuses on WHAT to execute and defines program logic, but not a detailed control flow. Declarative paradigm can be further classified into:

a) Logical Programming Paradigm: Logical programming paradigm is based on formal logic, which refers to a set of sentences expressing facts and rules about how to solve a problem

b) Functional Programming Paradigm: Functional programming is a programming paradigm where programs are constructed by applying and composing functions.

c) Database Programming Paradigm: Database programming model is used to manage data and information structured as fields, records, and files. be created or added to the existing codebase as needed.

Programming Paradigms



Why do we need a Constructor in Oops?

Creating an object without a constructor is like buying a car with no color or we will color it after buying. Using Accessor and Mutators functions is philosophically wrong. Therefore the constructor must be there for initializing the object. At the time of placing the order I wanted a white color Scorpio or fortuner etc.

So the purpose of a constructor is to create a new object and set values for any existing object properties.

A constructor in C++ is a special method that is automatically called when an object of a class is created. To create a constructor, use the same name as the class, followed by parentheses ().

There should not be a return type of constructor.



```

1  //Accessor and Mutator
2  #include<bits/stdc++.h>
3  using namespace std;
4  class Marker{
5  private:
6      string color;
7      string nature;
8  public:
9      //Default Constructor or Non - parameterized Constructor
10     Marker(){
11         color = "Red";
12         nature = "Temporary";
13     }
14     //Parameterized Constructor
15     Marker(string color = "Red", string nature = "Temporary"){
16         this->color = color;
17         this->nature = nature;
18     }
19     //Copy Constructor
20     Marker(Marker &m){
21         this->color = m.color;
22         this->nature = m.nature;
23     }
24     pair<string ,string> getter(){
25         return {color,nature};
26     }
27 };
28 int main(){
29     Marker m1("Blue","Temporary"), m2("Black","Permanent");
30     //Marker m3;
31     //When we calling m3 without passing any thing then
32     //Error - call of overloaded 'Marker()' is ambiguous, so either comment default constructor
33     //Or Pass Atleast One Arguments
34     Marker m3(m2);
35     cout<<"Marker M1 - Color "<<m1.getter().first<<" Nature "<<m1.getter().second<<endl;
36     cout<<"Marker M2 - Color "<<m2.getter().first<<" Nature "<<m2.getter().second<<endl;
37     cout<<"Marker M3 - Color "<<m3.getter().first<<" Nature "<<m3.getter().second;
38     return 0;
39 }

```

```
≡ output.txt X
≡ output.txt
1  Marker M1 - Color Blue Nature Temporary
2  Marker M2 - Color Black Nature Permanent
3  Marker M3 - Color Black Nature Permanent
```

Deep Copy Constructor :-

```
1 //Deep Copy Constructor
2 #include<bits/stdc++.h>
3 using namespace std;
4 class Test{
5 private:
6     int a;
7     int *p;
8 public:
9     Test(int x){
10         a = x;
11         p = new int[5];
12     }
13     Test(Test &t){
14         this->a = t.a;
15         //this->p = t.p; // This is wrong.
16         this->p = new int[5]; //here we called deep copy constructor.
17     }
18 };
```

There are three types of constructors:

- Default / Non Parameterized Constructor - With no parameters. C++ says that a default constructor is automatically provided by the compiler.
- Parameterized Constructor - With Parameters, Create a new instance of a class and also pass arguments simultaneously.
- Copy Constructor - Which creates a new object as a copy of an existing object.

Type of Function inside Class :-

- Constructor
- Accessor
- Mutator
- Facilitators
- Enquiry/Inspector
- Destructor

The using Directive :-

A C++ program can be divided into different namespaces. A namespace is a part of the program in which certain names are recognized; outside of the namespace they're unknown.

The directive **using namespace std;**

says that all the program statements that follow are within the std namespace. Various program components such as cout are declared within this namespace.

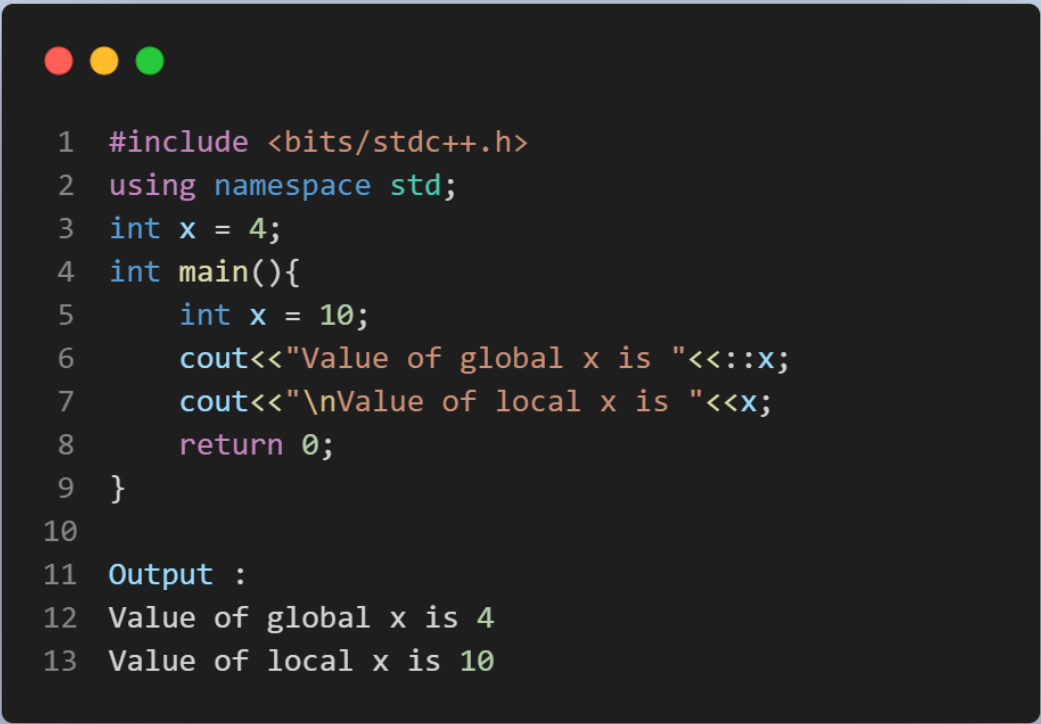
If we didn't use the using directive, we would need to add the std name to many program elements.

For example, in the FIRST program we'd need to say

```
std::cout << "Every age has a language of its own.";
```

Scope Resolution Operators (::) - it is used for the following purposes.

1. To access a global variable when there is a local variable with same name:



```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int x = 4;
4  int main(){
5      int x = 10;
6      cout<<"Value of global x is "<<::x;
7      cout<<"\nValue of local x is "<<x;
8      return 0;
9  }
10
11 Output :
12 Value of global x is 4
13 Value of local x is 10
```

2.To Define a Function Outside the class.

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  class A{
4  private:
5      int a,b;
6  public:
7      A(int x, int y){
8          a=x,b=y;
9      }
10     int Multiplication();
11 };
12 int A::Multiplication(){
13     return a*b;
14 }
15 int main(){
16     A obj(5,6);
17     cout<<"The Multiplication is : "<<obj.Multiplication();
18     return 0;
19 }
20
21 Output :
22 The Multiplication is : 30
```

3.To access a class's static variables.

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  class Test{
4      static int x;
5  public:
6      static int y;
7      void func(int x){
8          cout << "Value of static x is " << Test::x;
9          cout << "\nValue of local x is " << x;
10     }
11 };
12 // Static Member we can access directly without creating object.
13 int Test::x = 1;
14 int Test::y = 2;
15
16 int main(){
17     Test obj;
18     int x = 3 ;
19     obj.func(x);
20     cout << "\nTest::y = " << Test::y;
21     return 0;
22 }
23
24 Output:
25 Value of static x is 1
26 Value of local x is 3
27 Test::y = 2
28

```

4. In Case of multiple Inheritance: if the same variable name exists in two ancestor classes, we can use scope resolution operator to distinguish.



```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class A{
4  protected:
5      int x;
6  public:
7      A() { x = 10; }
8  };
9  class B{
10 protected:
11     int x;
12 public:
13     B() { x = 20; }
14 };
15
16 class C: public A, public B{
17 public:
18     void fun(){
19         cout << "A's x is " << A::x;
20         cout << "\nB's x is " << B::x;
21     }
22 };
23 int main(){
24     C c;
25     c.fun();
26     return 0;
27 }
28
29 Output:
30 A's x is 10
31 B's x is 20
```

5. Refer to a class inside another class :- refer to a nested class using the scope resolution operator.

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class outside{
4  public:
5      int x;
6      class inside {
7      public:
8          int x;
9          static int y;
10         int foo();
11     };
12 };
13 int outside::inside::y = 5;
14 int main(){
15     outside A;
16     outside::inside B;
17 }
18
```

Inline Function :-

When a function is declared inline, the compiler places a copy of that specific function's code at each point where the function is called at compile time.

When the function is declared outside the class with inline references then this can become overhead if the execution time of function is less than the switching time from the caller function to called function (callee).

Note :- **Inlining is only a request to the compiler, not a command.**

Compiler can ignore the request for inlining.

Compiler may not perform inlining in such circumstances:

- If the function contains a loop.
- If a function contains static Variables
- If a function is recursive in nature.
- If a function return type is other than void
- If a function contains switch or goto statements.

```
1  #include <iostream>
2  using namespace std;
3  class operation
4  {
5      int a,b,add,sub,mul;
6      float div;
7  public:
8      void get();
9      void sum();
10     void difference();
11 };
12 inline void operation :: get(){
13     cout << "Enter first value:";
14     cin >> a;
15     cout << "Enter second value:";
16     cin >> b;
17 }
18 inline void operation :: sum(){
19     add = a+b;
20     cout << "Addition of two numbers: " << a+b << "\n";
21 }
22 inline void operation :: difference(){
23     sub = a-b;
24     cout << "Difference of two numbers: " << a-b << "\n";
25 }
26 int main(){
27     cout << "Program using inline function\n";
28     operation s;
29     s.get();
30     s.sum();
31     s.difference();
32     return 0;
33 }
34
```

This Pointer :- this is a keyword that refers to the current instance of the class.

There can be 3 main uses of "this" keyword :

- It can be used to declare indexers.
- It can be used to refer to the current class instance variable.
- It can be used to pass the current object as a parameter to another method.

```
1  #include <iostream>
2  using namespace std;
3  class Node{
4  private:
5      int data;
6      Node* next;
7  public:
8      Node(int x){
9          this->data = x;
10         this->next = NULL;
11     }
12 };
13 int main(){
14     Node* n = new Node(5);
15     return 0;
16 }
```


Structure	Class
-----------	-------

A Structure is defined with the struct keyword	A class is defined with the class keyword
All the member of a structure are public by default	All the members of a class are private by default
Structure can't be inherit	Class can be inherit
A structure contains only data member	A class contain both data member and member function
There is no data hiding features	Classes having data hiding features by using access specifiers(public, private, protected)

Operator Overloading :- we can make operators work for user-defined classes. This means C++ has the ability to provide the operators with a special meaning for a data type, this ability is known as operator overloading.

For example, we can overload an operator '+' in a class like String so that we can concatenate two strings by just using +. Other example classes where arithmetic operators may be overloaded are Complex Numbers, Fractional Numbers, Big Integer, etc.

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  class Complex{
4  private:
5      int real;
6      int img;
7  public:
8      Complex(int r = 0, int i = 0){
9          real = r;
10         img = i;
11     }
12     void display(){
13         cout << real << " + i" << img << endl;
14     }
15     Complex add(Complex c){
16         Complex temp;
17         temp.real = real + c.real;
18         temp.img = img + c.img;
19         return temp;
20     }
21 };
22 int main(){
23     Complex c1(5, 3), c2(10, 5), c3;
24     c3 = c1.add(c2);
25     c3.display();
26 }
27
28 //Output
29 15 + i8
```

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  class Complex{
4  private:
5      int real;
6      int img;
7  public:
8      Complex(int r = 0, int i = 0){
9          real = r;
10         img = i;
11     }
12     void display(){
13         cout << real <<" + i" << img << endl;
14     }
15     Complex operator +(Complex c){
16         Complex temp;
17         temp.real = real + c.real;
18         temp.img = img + c.img;
19         return temp;
20     }
21     Complex operator -(Complex c){
22         Complex temp;
23         temp.real = real - c.real;
24         temp.img = img - c.img;
25         return temp;
26     }
27 };
28 int main(){
29     Complex c1(5, 3), c2(10, 5), c3, c4;
30     c3 = c2 + c1;
31     c4 = c2 - c1;
32     c4.display();
33     c3.display();
34 }
35
36 //Output
37 5 + i2
38 15 + i8
```

Operator Overloading using friend function :-

```
1 // Operator Overloading using Friend Functions
2 #include <iostream>
3 using namespace std;
4 class Complex{
5 private:
6     int real;
7     int img;
8 public:
9     Complex(int r = 0, int i = 0){
10         real = r;
11         img = i;
12     }
13     void display(){
14         cout << real <<" + i"<< img << endl;
15     }
16     friend Complex operator+(Complex c1, Complex c2);
17 };
18 Complex operator+(Complex c1, Complex c2){
19     Complex temp;
20     temp.real = c1.real + c2.real;
21     temp.img = c1.img + c2.img;
22     return temp;
23 }
24 int main(){
25     Complex c1(5, 3), c2(10, 6), c3;
26     c3 = c1 + c2;
27     // also written like this
28     // c3 = operator+(c1,c2);
29     c3.display();
30 }
31
32 // Output
33 15 + i9;
```

Ostream Operators Overloading :-

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  class Complex{
4  private:
5      int real;
6      int img;
7  public:
8      Complex(int r, int i){
9          real = r;
10         img = i;
11     }
12     friend ostream & operator<<(ostream &o, Complex &c);
13 };
14 ostream & operator<<(ostream &out, Complex &c){
15     out<<c.real<<" + i"<<c.img<<endl;
16     return out;
17 }
18 int main(){
19     Complex c1(4,6);
20     cout<<c1;
21     operator<<(cout,c1);
22     return 0;
23 }
24
25 // Output
26 4 + i6
27 4 + i6
```

Eg - Addition of Rational Numbers



```

1 // Program with a Class Rational Number
2 #include <iostream>
3 using namespace std;
4 class Rational{
5 private:
6     int p;
7     int q;
8 public:
9     Rational(){
10         p = 1;
11         q = 1;
12     }
13     Rational(int p, int q){
14         this->p = p;
15         this->q = q;
16     }
17     Rational(Rational &r){
18         this->p = r.p;
19         this->q = r.q;
20     }
21     int getP() { return p; }
22     int getQ() { return q; }
23     void setP(int p){
24         this->p = p;
25     }
26     void setQ(int q){
27         this->q = q;
28     }
29     Rational operator+(Rational r){
30         Rational t;
31         t.p = this->p * r.q + this->q * r.p;
32         t.q = this->q * r.q;
33         return t;
34     }
35     friend ostream &operator<<(ostream &os, Rational &r);
36 };
37 ostream &operator<<(ostream &os, Rational &r){
38     os << r.p << "/" << r.q;
39     return os;
40 }
41 int main(){
42     Rational r1(3, 4), r2(2, 5), r3;
43     r3 = r1 + r2;
44     cout << "Sum of " << r1 << " and " << r2 << " is " << r3 << endl;
45 }
46
47 // Output
48 Sum of 3/4 and 2/5 is 23/20

```